If we search for *Peter* and *Peppers*, for example, we expect the document to match our search criteria. It would be difficult to match search criteria if text was dumped in as is without being converted to tokens.

While we've separated the words based on whitespace in the example in table 3.1, a few tokenizer variants available in Elasticsearch can help us extract tokens based on digits, nonletters, stop words, and so on; we are not limited to whitespace. For example, suppose we want to search our data in other ways. Continuing the same example, users could use any of the following search text:

- "Who is Peter Piper?"
- "What did Peter pick?"
- "Has Peter picked pickled peppers?"
- "Peter Pickle Peppers"
- "Chili pickles" (our search will return this document on this query)

During tokenization, Elasticsearch persists the tokens (individual words) but doesn't enhance, enrich, or transform them. That is, the tokens are persisted as is. If we search for "pickled peppers", we'll probably get this Peter Piper document, although the relevance score (which tells us how closely the result of a search matches the query criteria) may be a bit low. However, searching for "chili capsicum" may yield no results (the other name for a pepper is capsicum). Unless we have enriched tokens, there's no easy way to answer the search for such text—and that's why we have another process called normalization that works on these tokens.

Normalization helps build a rich user experience by creating additional data around tokens. It is a process of reducing (stemming) tokens to root words or creating synonyms for them. For example:

- The `peppers` token can be stemmed to create alternate words like *capsicum*.
- The `Piper` token can be stemmed to produce *Bagpiper*.

Normalization can also help build a list of synonyms for tokens, again enriching the user's search experience. For instance, a list of synonyms for *work* may include *working*, *office work*, *employment*, or *job*; and *authoring*, *authored*, and *authored by* all relate to *author*.

Along with the tokens, root words, synonyms, and so on are stored in an advanced data structure called an *inverted index*. Let's take an example of a single word, *vaccine*. It can be analyzed and stored in the inverted index against several root words or synonyms like *vaccination*, *vaccinated*, *immunization*, *immune*, *inoculation*, *booster jab*, and so forth.

When a user searches for *immunization*, for example, documents with *vaccine* or related words may pop up, as these words are related to each other. If you search for "where can I get immunization for covid*"* on Google, there is a high likelihood that your results will be *vaccine* related. When I tried this search, Google returned the expected results (see figure 3.2).